National University of Singapore

Department of Mechanical Engineering

ME5411 Robot Vision and AI

Group Project Report

Academic Year 2026/2027

Group 17

Name	Student Number
LI JIHAN	A0350821M
HU ZHENG	Student Number 2
XU DARUI	Student Number 3
ZHAN CHENG	Student Number 3

Submission date: 20 November 2026

Contents

1 Introduction **1**
1.1 Processing and experimental workflows . . . 1
2 Task 1: Original Image and Contrast Enhancement **3**
2.1 Objective and Input Image . . . 3
2.2 Implemented Enhancement Methods . . . 3
2.3 Evaluation Protocol . . . 3
2.4 Result Analysis . . . 5
2.5 Selected Output for Later Tasks . . . 5
3 Task 2: Image Smoothing with Averaging Filters **5**
3.1 Objective . . . 5
3.2 Implemented Averaging Filter . . . 5
3.3 Evaluation Protocol . . . 6
3.4 Result Analysis . . . 6
4 Task 3: Sub-image Extraction **7**
4.1 Objective and Input Image . . . 7
4.2 Row Detection Method . . . 7
4.3 Extraction Result . . . 8
4.4 Verification . . . 8
5 Task 4: Binary Image Conversion **9**
5.1 Objective . . . 9
5.2 Binarization Methods Compared . . . 9
5.3 Selected Method and Analysis . . . 10
6 Task 5: Character Outline Detection **10**
6.1 Objective . . . 10
6.2 Contour Following Method . . . 11
6.3 Results . . . 11
6.4 Analysis . . . 12
7 Task 6: Character Segmentation and Labelling **13**
7.1 Objective . . . 13
7.2 Segmentation Method . . . 13
7.3 Results . . . 13
7.4 Why Component Labelling Rather than Projection Splitting . . . 13
7.5 Analysis . . . 15
8 Task 7: Character Classification **15**
8.1 Objective and experimental design . . . 15
8.2 Data split and leakage control . . . 15
8.3 Common preprocessing . . . 16
8.4 Subtask 7-1: CNN classifier . . . 16
8.5 Subtask 7-2: MLP one-hot regression . . . 16
8.6 Hyperparameter and preprocessing investigation . . . 16
8.7 Held-out results and efficiency . . . 18
8.8 Inference on the Figure-1 character row . . . 18
8.9 Reproduction and environment . . . 20
9 Implementation and Reproduction Guide **20**
10 Discussion and Conclusion **22**
References **22**

List of Figures

1.1 Selected Tasks 1–6 workflow. The dashed arrow transfers the selected filter setting; Task 3 applies that filter to Task 1's output. Task 5 and Task 6 are separate branches of the Task 4 binary image. . . . 2
1.2 Task 7 experimental workflow. The outer test set and Figure 1 are excluded from optimisation and model selection. Target geometry alignment uses only statistics of the inner training images. . . . 2
2.1 Original image and contrast enhancement results. All grayscale images use the same display range of 0–255. . . . 4
2.2 Histograms of the grayscale baseline and the three enhancement methods. . . . 4
3.1 The enhanced image and the box-filtered results for five filter sizes. All images use the same display range of 0–255. . . . 6
3.2 Zoomed view of the `HD4` characters. Larger kernels remove more background texture but progressively blur the stroke edges. . . . 6
4.1 Detected `HD44780A00` row with the extraction rectangle drawn in red. The rectangle contains a 10-pixel safety margin around the refined character extent. . . . 8
4.2 The 899×148 sub-image containing the complete `HD44780A00` row, extracted with the box $(x, y) \in [52, 951) \times [194, 342)$. . . . 9
5.1 The Task 3 sub-image and the three binarization candidates. White pixels are foreground. . . . 10
6.1 Traced outlines overlaid on the grayscale sub-image. Solid lines are outer boundaries; dashed lines are hole boundaries. . . . 11
6.2 Outline detection outputs. . . . 12
7.1 Segmented characters labelled 1–10 in reading order. . . . 14
7.2 Bounding boxes of the ten segmented characters. . . . 14
7.3 The ten extracted character crops (labels 1–10). . . . 14
7.4 Character groups found on the global-Otsu binary (7 groups) and on the selected adaptive binary (10 groups). Merged neighbours are visible as boxes that span two characters. . . . 14
8.1 Validation accuracy and loss for the resolution and learning-rate investigation. . . . 18
8.2 Confusion matrices on the duplicate-isolated held-out test set. . . . 19
8.3 The original inverted grayscale inputs (top) retain background texture. Fixed binary foreground and training-median geometry alignment (bottom) produce cleaner, centred inputs without fitting on Figure 1. . . . 19

List of Tables

2.1 Task 1 quantitative comparison. . . . 4
3.1 Task 2 quantitative comparison of the averaging-filter sizes. . . . 7
5.1 Task 4 binarization comparison. . . . 10
6.1 Per-character outline statistics. "Expected" is the ground-truth glyph in `HD44780A00`. 12
7.1 Segmentation result. Bounding boxes are given in sub-image pixel coordinates. 15
8.1 Complete single-network validation grid. Bold configurations are selected for final fitting. CNN loss is cross-entropy; MLP loss is one-hot MSE, so the two loss scales should not be compared directly. . . . 17
8.2 Shared training settings and the final-fit schedule. . . . 17
8.3 Final held-out test performance and measured efficiency. Parameter counts are per member; training time covers the complete ensemble. . . . 18
8.4 Figure 1 predictions, with ten ordered positions made explicit. Accuracy is measured on this single reference row after inference. . . . 19

1 Introduction

Recognising characters printed on microchips supports component identification and visual inspection, but reliable recognition depends on both the image processing pipeline and the classifier. The supplied 990×367 chip photograph contains the target row `HD44780A00`, with bright strokes on a dark, non-uniform surface. Ring-like texture and speckle noise overlap the character intensity range, while weak strokes and narrow gaps make global thresholding and character separation difficult. This project investigates how to preserve the character structure through enhancement, smoothing, segmentation, and supervised recognition, and how decisions at each stage affect the final result.

Tasks 1–6 develop the image-processing pipeline and compare alternative methods using intermediate images and quantitative measurements. Percentile stretching, gamma correction, and histogram equalisation are evaluated before averaging filters of several sizes are tested. The selected 2–98% percentile stretch and 5×5 smoothing support projection-based extraction of the character row. Global Otsu and local adaptive thresholding are then compared: the adaptive method preserves ten separate character regions, whereas global thresholding merges neighbouring glyphs. Moore-neighbour contour following extracts the character outlines, and two-pass connected-component labelling with area filtering produces ten labelled crops in reading order. These experiments connect the quality of the earlier preprocessing stages to the later segmentation outcome.

Task 7 compares a convolutional neural network (CNN) with a fully connected multilayer perceptron (MLP) trained as a supervised regression model. The MLP regresses seven-dimensional one-hot targets with mean-square error, and three independently trained members are combined by averaging their output scores. The 7,112 labelled images in `dataset_2026`, covering the classes `0`, `4`, `7`, `8`, `A`, `D`, and `H`, are divided into 75% training and 25% test sets. Exact duplicate images remain within one partition, and model selection uses validation data drawn only from the training pool. Both networks use input standardisation and batch normalisation; input resolution and learning rate are investigated. Accuracy, macro-F1, confusion matrices, parameter counts, and measured training and inference times provide the effectiveness and efficiency comparison. The chip photograph and its segmented characters are reserved for inference after training.

The selected CNN and MLP ensemble achieve held-out test accuracies of 99.33% and 98.71%, respectively. Direct grayscale inference on the photographed row identifies one of ten characters with the CNN and nine with the MLP. A further class-independent preprocessing experiment uses the segmented binary foreground and centres it at the median foreground dimensions estimated only from the tuning-training images. With the same trained models, this improves the target-row results to eight of ten and ten of ten characters, respectively. This ten-character check illustrates sensitivity to image appearance; it is separate from the larger held-out test and does not establish general target-domain accuracy. Tasks 1–6 describe the processing stages, section 8 presents the learning experiments, and section 9 explains the implementation and reproduction procedure.

1.1 Processing and experimental workflows

The two workflows show the image-processing dependencies and the separation between training, model selection, evaluation, and inference. figure 1.1 follows the selected image-processing methods; the Task 5 contour branch and Task 6 segmentation branch share the same binary input. figure 1.2 shows that Figure 1 has no path into training or model selection.

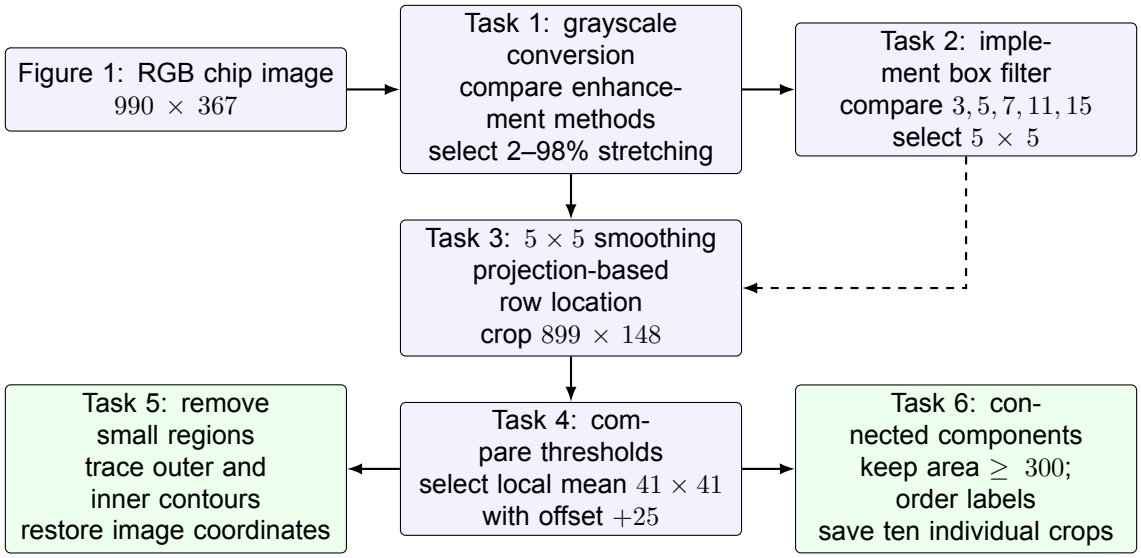


Figure 1.1: Selected Tasks 1–6 workflow. The dashed arrow transfers the selected filter setting; Task 3 applies that filter to Task 1's output. Task 5 and Task 6 are separate branches of the Task 4 binary image.

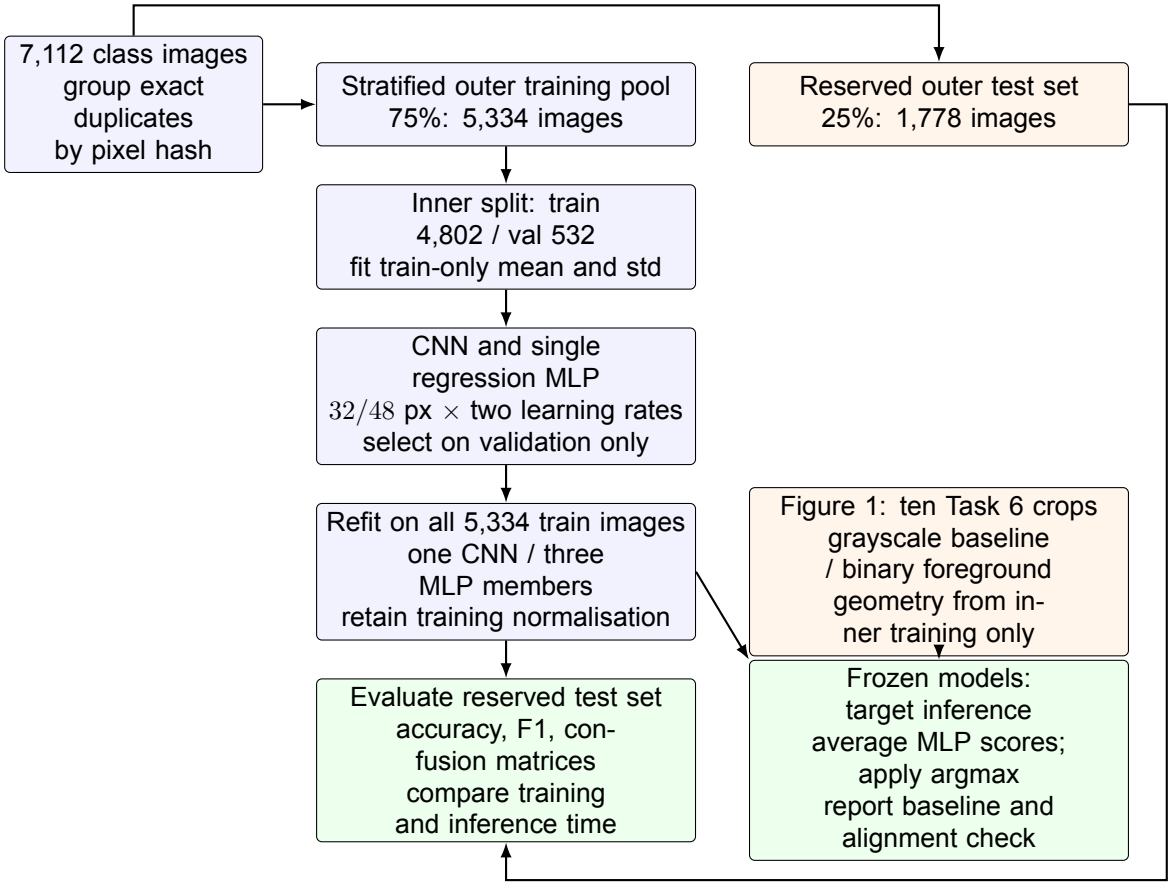


Figure 1.2: Task 7 experimental workflow. The outer test set and Figure 1 are excluded from optimisation and model selection. Target geometry alignment uses only statistics of the inner training images.

2 Task 1: Original Image and Contrast Enhancement

2.1 Objective and Input Image

Task 1 requires displaying the original chip image, applying contrast enhancement, and commenting on the result. The input image is an RGB bitmap of size 990×367 pixels. Its RGB pixel range is 41–255, while the grayscale luminance range is approximately 41.91–252.31. The grayscale median is 82.91, and the 2nd and 98th percentiles are 63.63 and 156.00 respectively.

The original image in figure 2.1 shows bright characters on a dark, non-uniform background. The main difficulty is not low brightness alone: the background contains dense ring-like texture and bright speckle noise. A useful enhancement method must therefore improve the character-to-background separation without making the background texture look like character strokes.

2.2 Implemented Enhancement Methods

The enhancement operators follow standard digital image-processing definitions [1]; their experiment code was implemented in Python in `task_1/task_1.py`. The RGB image was first converted to grayscale luminance by

$$Y = 0.299R + 0.587G + 0.114B. \tag{2.1}$$

The grayscale image is the baseline for comparison. Three contrast enhancement methods were tested on the same baseline:

1. **2–98% percentile stretching.** The 2nd percentile is mapped to 0 and the 98th percentile is mapped to 255:
$$I_{\text{out}} = \text{clip}\left(255\frac{I - p_2}{p_{98} - p_2},\, 0,\, 255\right). \tag{2.2}$$
This ignores extreme dark and bright outliers and uses the available grayscale range more aggressively.
2. **Gamma correction.** A power-law transform with $\gamma = 1.5$ was applied:
$$I_{\text{out}} = 255\left(\frac{I}{255}\right)^{1.5}. \tag{2.3}$$
Since $\gamma > 1$, mid-level background pixels are darkened. This tests whether suppressing the background improves character visibility.
3. **Global histogram equalization.** A 256-level cumulative distribution function mapping was applied to spread the global grayscale histogram. This is included as a strong global enhancement baseline.

For a fair visual comparison, all grayscale images in figure 2.1 are displayed with the same scale, $v_{\text{min}} = 0$ and $v_{\text{max}} = 255$. No method is given a separate automatic display stretch.

2.3 Evaluation Protocol

The result was evaluated both visually and quantitatively. Visual evaluation checks whether the characters, especially `T12` and `HD44780A00`, become clearer without excessive background amplification. Quantitative evaluation uses three values:

- **CNR**: contrast-to-noise ratio between fixed character-stroke patches and fixed nearby background patches,
$$\text{CNR} = \frac{|\mu_c - \mu_b|}{\sqrt{\sigma_c^2 + \sigma_b^2}}. \tag{2.4}$$

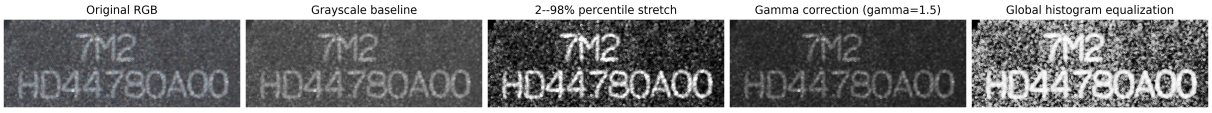


Figure 2.1: Original image and contrast enhancement results. All grayscale images use the same display range of 0–255.

Larger CNR means better separation between character pixels and background pixels in the selected regions.

- **Otsu separability**: normalized between-class variance from Otsu's thresholding criterion. A larger value suggests that the grayscale histogram is easier to split into two classes, but it does not guarantee that the foreground class corresponds only to characters.
- **Saturation ratio**: percentage of pixels clipped to the endpoints 0 or 255. A large value indicates loss of grayscale detail.

Table 2.1: Task 1 quantitative comparison.

Method	CNR	Otsu separability	Saturation ratio
Grayscale baseline	0.6933	0.6941	0.00%
2–98% percentile stretch	**0.7013**	0.7181	4.22%
Gamma correction, $\gamma = 1.5$	0.6948	0.7077	0.00%
Global histogram equalization	0.5669	**0.7410**	0.41%

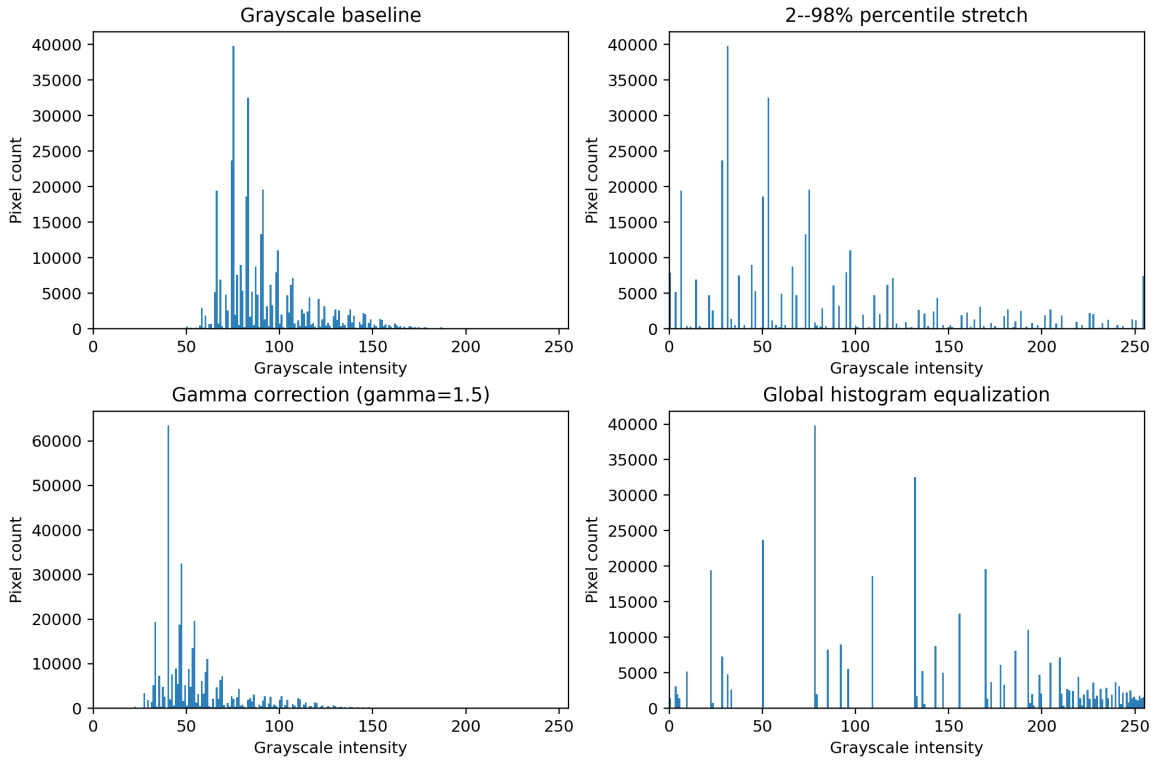


Figure 2.2: Histograms of the grayscale baseline and the three enhancement methods.

2.4 Result Analysis

The grayscale baseline preserves the original information, but the characters are only weakly separated from the textured background. Its CNR is 0.6933 and its Otsu separability is 0.6941, which provides the reference level for the enhancement methods.

The 2–98% percentile stretch gives the most useful result for this task. It raises CNR from 0.6933 to 0.7013 and raises Otsu separability from 0.6941 to 0.7181. In figure 2.1, the strokes in `T12` and `HD44780A00` become visibly brighter and easier to locate. The trade-off is a 4.22% saturation ratio, caused by clipping the lowest and highest tails of the intensity distribution. This clipping is acceptable here because it mainly increases character visibility; it does not dominate the image with false foreground structures.

Gamma correction with $\gamma = 1.5$ is conservative. It slightly improves CNR to 0.6948 and Otsu separability to 0.7077 without endpoint clipping. However, the improvement is small because the transform darkens both background texture and weaker character strokes. It does not separate the characters as clearly as percentile stretching.

Global histogram equalization is not selected despite its highest Otsu separability of 0.7410. The visual result shows that it strongly amplifies the ring-like background texture and bright speckles. Its CNR also drops to 0.5669, which means the selected character and background patches become less separable than in the original grayscale image. This is a clear example where a wider or more separable global histogram does not imply better character readability. For later binary conversion and segmentation, this result would likely create many false foreground regions.

2.5 Selected Output for Later Tasks

The selected Task 1 output is the 2–98% percentile-stretched image saved in `task_1/outputs/`. It provides the best balance among the tested methods: character strokes become clearer, the measured CNR is the highest, and the background is not amplified as severely as in global histogram equalization. This output is therefore the preferred input candidate for the following smoothing, thresholding, outline detection, and segmentation stages.

3 Task 2: Image Smoothing with Averaging Filters

3.1 Objective

Task 2 requires implementing a 5×5 averaging filter, applying it to the image, trying additional filter sizes, and comparing the smoothing results. Because Task 1 selected the 2–98% percentile-stretched image as the preferred input for the downstream stages, every smoothing experiment here operates on that enhanced grayscale image.

3.2 Implemented Averaging Filter

An averaging (box) filter [1] of size $s \times s$ replaces each pixel with the mean of the $s \times s$ neighbourhood centred on it:

$$I_{\text{out}}(x, y) = \frac{1}{s^2} \sum_{i=-r}^{r} \sum_{j=-r}^{r} I_{\text{in}}(x + i, y + j), \qquad r = \frac{s-1}{2}. \tag{3.1}$$

The implementation exploits the separability of the box kernel and computes the sliding sums with a summed-area table, so the running time is $O(hw)$ independent of s rather than $O(hws^2)$ for a naive sliding window. The filter is implemented from first principles with NumPy only in `common.box_filter`; boundary pixels are handled by replicating the edge values. The required

5×5 size was implemented together with four additional odd sizes, 3×3, 7×7, 11×11, and 15×15, to make the smoothing–detail trade-off visible.

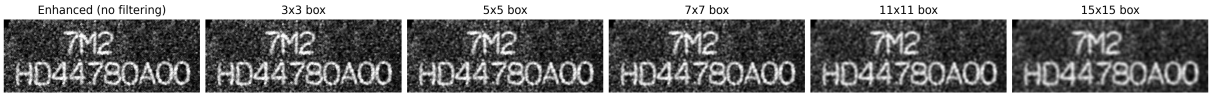


Figure 3.1: The enhanced image and the box-filtered results for five filter sizes. All images use the same display range of 0–255.

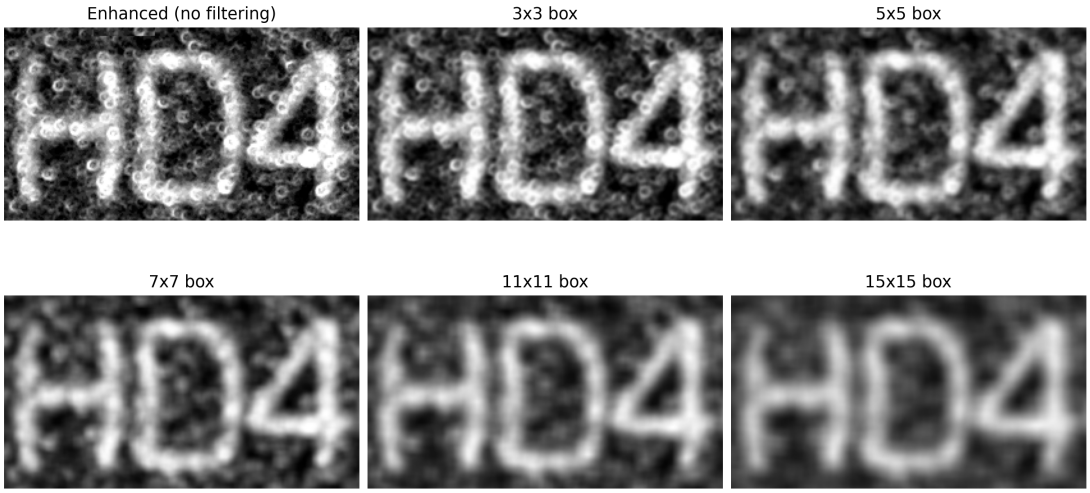


Figure 3.2: Zoomed view of the `HD4` characters. Larger kernels remove more background texture but progressively blur the stroke edges.

3.3 Evaluation Protocol

The smoothing results were compared both visually and quantitatively. Three values were measured on fixed image regions so that all filter sizes are judged identically:

- **Background standard deviation**: the pooled standard deviation of five fixed character-free background patches. A smaller value means the ring-like texture and speckle noise were suppressed more strongly.
- **Foreground gradient**: the mean gradient magnitude over five fixed character-stroke patches. A smaller value means the stroke edges were blurred more, i.e. more detail was lost.
- **PSNR**: peak signal-to-noise ratio relative to the unfiltered enhanced image, measuring how much the whole image changed.

3.4 Result Analysis

The measurements in table 3.1 show a clear monotonic trade-off. As the kernel grows, the background standard deviation falls from 33.42 to 16.36, confirming stronger noise suppression, but the foreground gradient also falls from 20.18 to 6.75, meaning the character edges are increasingly blurred. The 3×3 filter is too weak: it reduces the background noise by only

Table 3.1: Task 2 quantitative comparison of the averaging-filter sizes.

Filter	Background std.	Foreground gradient	PSNR (dB)
None (enhanced baseline)	33.42	20.18	∞
3×3 box	29.56	14.75	28.50
5×5 box	26.46	11.00	24.03
7×7 box	24.09	9.44	21.95
11×11 box	19.91	7.86	20.17
15×15 box	16.36	6.75	18.90

11.5% while the largest 15×15 filter removes 51.0% of the noise but erases 66.5% of the edge sharpness. In figure 3.2, the thin strokes of the characters visibly soften at 11×11 and start to blend with the background at 15×15; such damage would later break or merge character strokes during thresholding.

The required 5×5 filter is the best compromise for this image. It reduces the background standard deviation by 20.8% while retaining 54.5% of the edge gradient magnitude, so isolated speckles are suppressed without destroying the thin character strokes. Its PSNR of 24.03 dB confirms that the change is moderate. This 5×5 smoothing was therefore reused in Task 3 for row localization and in Task 4 before thresholding, keeping the image-processing pipeline consistent.

4 Task 3: Sub-image Extraction

4.1 Objective and Input Image

Task 3 requires extracting a sub-image that contains the complete character row

$$\texttt{HD44780A00} \tag{4.1}$$

The input is the 990×367 enhanced grayscale image selected in Task 1, the 2–98% percentile-stretched output saved under `task_1/outputs/`. The row of interest is the lower of the two text rows in the image; the upper row is the logo `T12`. The extraction must therefore not only crop the correct vertical band but also decide the horizontal extent so that none of the ten characters is cut off.

4.2 Row Detection Method

The row was located with projection profiles, which are computed entirely with NumPy in the Task 3 script `task_3/task_3.py`. The procedure has three stages.

4.2.1 Preprocessing for detection

The enhanced image is first smoothed with a 5×5 box filter (the filter implemented in Task 2) so that isolated bright speckles do not dominate the profiles. A binary mask is then obtained with Otsu's thresholding, whose threshold t^* maximises the between-class variance

$$t^* = \arg\max_t \; \omega_0(t)\,\omega_1(t)\,\big(\mu_0(t) - \mu_1(t)\big)^2 \tag{4.2}$$

where ω_0, ω_1 are the class probabilities and μ_0, μ_1 the class means on either side of threshold t. For the smoothed enhanced image the Otsu threshold is $t^* = 117$.

4.2.2 Coarse row band

The horizontal projection counts foreground pixels per image row,

$$R(y) = \sum_{x} B(x, y), \tag{4.3}$$

and the vertical projection counts foreground pixels per column,

$$C(x) = \sum_{y} B(x, y). \tag{4.4}$$

Contiguous rows with $R(y) > 25$ form candidate text bands. The lowest band below $y = 185$ is the required row, giving the coarse box $(x, y) \in [48, 942) \times [198, 336)$.

4.2.3 Band-adaptive refinement

The coarse box is looser than the true character extent because Otsu's mask also marks part of the bright ring-like background texture. The box is therefore refined inside the coarse band: the band is thresholded again at its 80th intensity percentile (measured here as 183.6), which lies above the textured background but below the character stroke cores. Row spans shorter than 40 pixels are discarded, and column spans separated by up to 30 pixels are merged so that the spaces between characters do not split the row. This yields the refined box $(x, y) \in [62, 941) \times [204, 332)$.

4.3 Extraction Result

A safety margin of 10 pixels is added on every side and clamped to the image bounds, giving the final extraction box $(x, y) \in [52, 951) \times [194, 342)$, i.e. a sub-image of 899×148 pixels. figure 4.1 shows the detected rectangle overlaid on the full enhanced image, and figure 4.2 shows the extracted sub-image in both its original RGB form and the enhanced grayscale form that continues into the later processing stages.

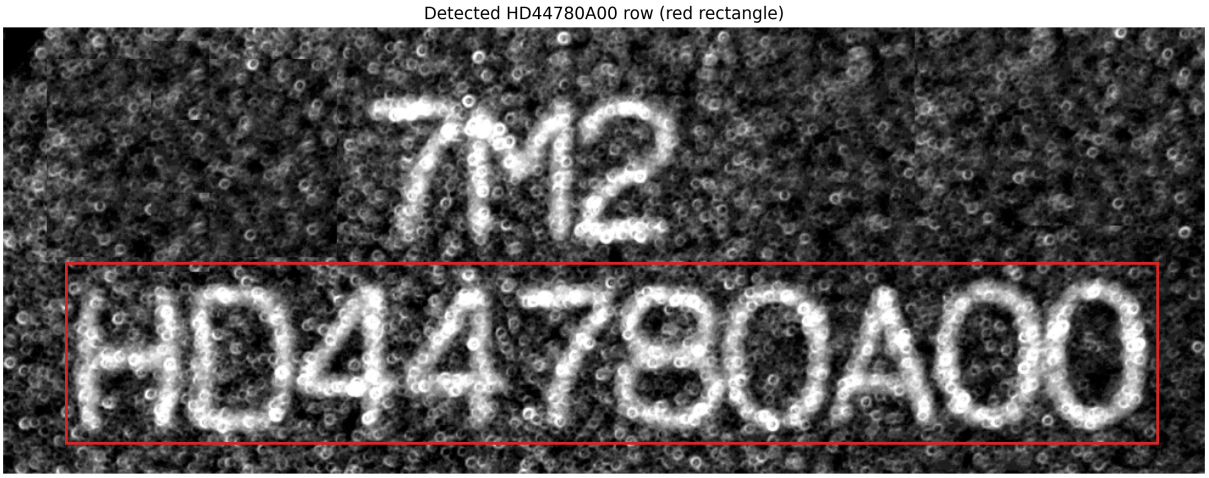
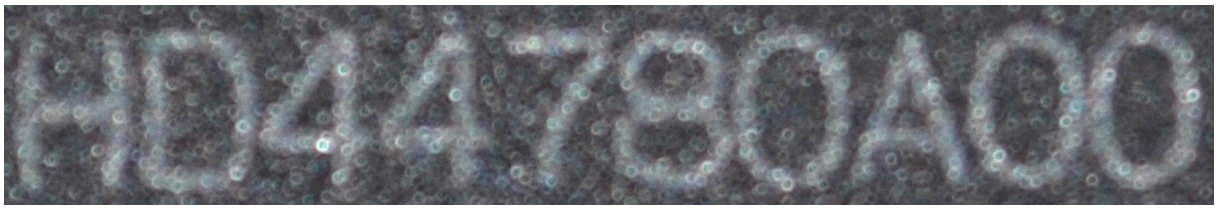


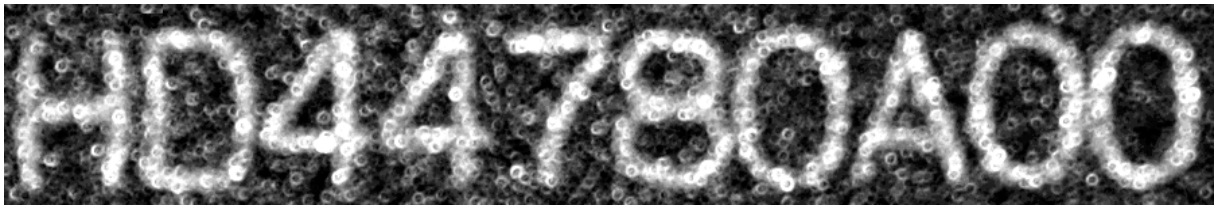
Figure 4.1: Detected `HD44780A00` row with the extraction rectangle drawn in red. The rectangle contains a 10-pixel safety margin around the refined character extent.

4.4 Verification

The result was verified at the connected-component level. Thresholding the 5×5-smoothed crop at intensity 140 and labelling the components recovers the ten characters as eight large



(a) Extracted sub-image, original RGB colour.



(b) Extracted sub-image, enhanced grayscale (Task 1 output).

Figure 4.2: The 899×148 sub-image containing the complete `HD44780A00` row, extracted with the box $(x, y) \in [52, 951) \times [194, 342)$.

stroke groups with areas between $2{,}756$ and $8{,}933$ pixels; every other component is smaller than 110 pixels. The labelled stroke groups span $x \in [60, 941]$ and $y \in [205, 341]$, which lies entirely inside the final extraction box. All ten characters `H`, `D`, `4`, `4`, `7`, `8`, `0`, `A`, `0`, `0` are therefore complete in the sub-image. The `T12` logo, whose lowest stroke ends at $y = 181$, is not included because the crop starts at $y = 194$.

5 Task 4: Binary Image Conversion

5.1 Objective

Task 4 requires converting the Task 3 sub-image of size 899×148 into a binary image and showing the intermediate results. Three thresholding schemes were compared, and the selected method keeps the ten characters as separate foreground components while rejecting most of the textured background.

5.2 Binarization Methods Compared

1. **Global Otsu thresholding.** Otsu's method [2] chooses the level T^* that maximizes the between-class variance of the intensity histogram:

$$T^* = \arg\max_T \; w_0(T)\, w_1(T) \big(\mu_0(T) - \mu_1(T)\big)^2, \qquad B(x, y) = I(x, y) > T^*, \tag{5.1}$$

   where w_k and μ_k are the weight and mean of each class. A single global level is simple, but it cannot adapt to the spatially varying background texture.
2. **Global Otsu after smoothing and morphological opening.** The sub-image is first smoothed with the 5×5 box filter from Task 2, then thresholded globally, and finally opened with a 3×3 structuring element (an erosion followed by a dilation) to delete small speckle components.
3. **Local adaptive mean thresholding.** The threshold becomes a slowly varying field based on the local mean over a 41×41 window:

$$T(x, y) = \mu_{41}(x, y) + 25, \qquad B(x, y) = I(x, y) > T(x, y). \tag{5.2}$$

   After 5×5 smoothing, a pixel is foreground only when it is more than 25 grey levels brighter than its local surroundings. This is well matched to bright characters on a dark,

locally textured background.

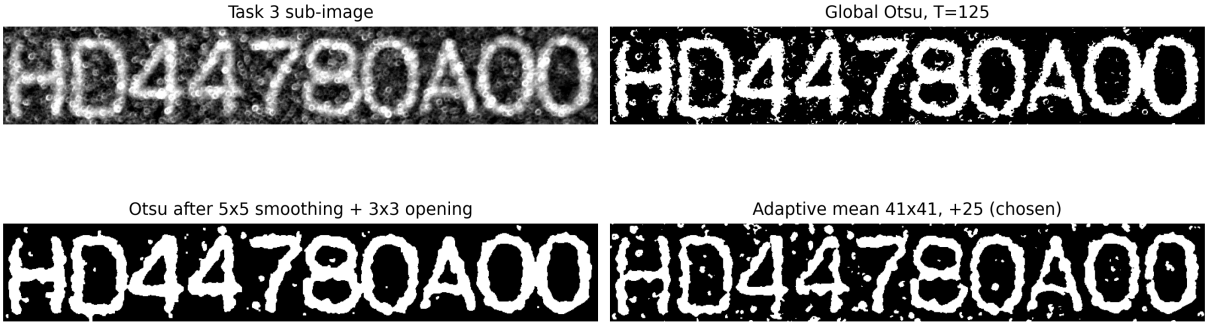


Figure 5.1: The Task 3 sub-image and the three binarization candidates. White pixels are foreground.

Table 5.1: Task 4 binarization comparison.

Method	Foreground (%)	Components	Character comp.	Max. noise (px)
Global Otsu, $T = 125$	34.00	296	7	131
Smoothed Otsu + open 3×3	34.31	32	7	135
Adaptive 41×41, $+25$ (chosen)	30.72	126	**10**	224

5.3 Selected Method and Analysis

Global Otsu thresholding at $T = 125$ performs poorly here. It labels 34.0% of the image as foreground because the bright ring texture crosses the single global level, producing 296 connected components, and its character strokes merge into only 7 character-sized components. Smoothing followed by opening reduces the component count to 32, but the characters remain merged and the largest speckles survive.

The adaptive mean threshold is selected as the final method. It labels 30.7% of the image as foreground and produces 126 components, of which exactly 10 have an area above 500 pixels; these ten components correspond one-to-one to the ten characters of `HD44780A00` as read left to right. The remaining 116 components are noise smaller than 225 pixels, which are easy to remove with an area criterion in Task 6. Unlike Otsu, the local criterion does not require one grey level to separate character from texture across the whole image; instead it marks only pixels that stand out against their own neighbourhood.

The selected binary image is saved as `task_4/outputs/task4_binary.png` and becomes the input for the outline detection and segmentation tasks.

6 Task 5: Character Outline Detection

6.1 Objective

Task 5 requires determining the outline(s) of the characters in the image and displaying them. The input is the binary image selected in Task 4 (`task4_binary.png`), in which the ten characters appear as bright strokes on a dark background. Because the binary image already separates

the characters from the background, outline detection reduces to following the border between foreground and background for every character-sized connected component.

6.2 Contour Following Method

The outlines were traced with Moore-neighbor contour following [3], implemented from first principles in `common.py`. For one 8-connected foreground component the algorithm works as follows.

1. Start at the top-leftmost foreground pixel b_0; the backtrace pixel c_0 is its western neighbour, which is background by construction.
2. Examine the eight neighbours of the current boundary pixel b_k in clockwise order, beginning just after the backtrace pixel c_k, until the first foreground neighbour is found; it becomes b_{k+1}, and the neighbour examined just before it becomes the new backtrace pixel c_{k+1}.
3. Repeat until the start pixel is re-entered from the same direction (Jacob's stopping criterion), which closes the contour.

The walk is clockwise around the component, so the outer boundary is visited in order and its pixels can be encoded as an 8-direction Freeman chain code, with direction 0 east, 1 north-east, 2 north, 3 north-west, 4 west, 5 south-west, 6 south, and 7 south-east. Characters such as `D`, `4`, `8`, `0`, and `A` also contain enclosed holes. The holes are the background regions inside a component that do not touch the image border; each hole is identified by labelling the inverted component and traced with the same follower, so the method returns the outer outline *and* every inner outline.

Tracing is performed on a padded crop of each character. Before plotting or exporting, a crop coordinate (r, c) is converted back to the grayscale sub-image coordinate $(r - p + y_0, c - p + x_0)$, where $p = 1$ is the padding and (y_0, x_0) is the crop origin. This places every outer and inner contour on its own character. The script verifies that all trace coordinates lie inside the corresponding bounding box, that outer trace pixels are foreground, and that hole trace pixels are background.

6.3 Results

figure 6.1 overlays the traced outlines on the grayscale sub-image: solid curves are outer boundaries and dashed curves are hole boundaries, with one colour per character. figure 6.2a shows the boundary pixels alone, and figure 6.2b zooms into the second 4, marking the trace start and the clockwise walking direction.

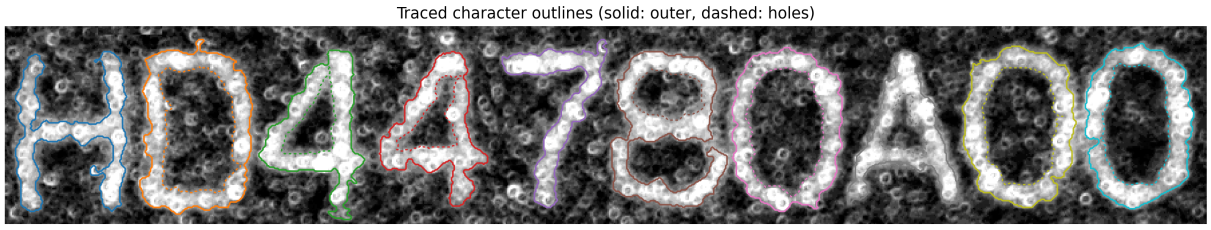
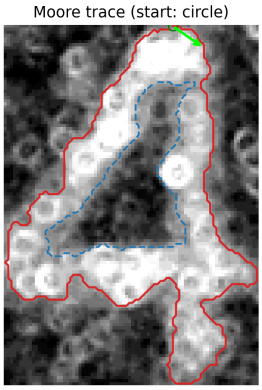


Figure 6.1: Traced outlines overlaid on the grayscale sub-image. Solid lines are outer boundaries; dashed lines are hole boundaries.



(a) Boundary pixels only.



(b) Zoom on the second `4`: start point and trace direction.

Figure 6.2: Outline detection outputs.

Table 6.1: Per-character outline statistics. “Expected” is the ground-truth glyph in `HD44780A00`.

Order	Expected	Outer length	Holes	Hole lengths
1	`H`	619	0	–
2	`D`	379	1	270
3	`4`	368	1	164
4	`4`	340	1	172
5	`7`	408	0	–
6	`8`	515	1	135
7	`0`	329	3	1, 2, 236
8	`A`	364	1	130
9	`0`	332	2	229, 3
10	`0`	325	1	230

6.4 Analysis

The follower returns a closed outer boundary for all ten characters; the total outer boundary length of the row is $3{,}979$ pixels. The hole counts agree with the topology of most glyphs: `H` and `7` have no holes, and `D`, `4`, and `A` each have one enclosed region.

Three details show where the binarization stage leaves its fingerprint on the outlines. First, the `8` yields one hole instead of the two expected for its two loops: in the thresholded image a stroke gap opens the bottom loop interior to the background, so only the top loop remains enclosed. Second, the first `0` reports two tiny holes of 1–2 pixels beside its main hole, and the middle `0` one of 3 pixels: these are spurious holes created by small stroke gaps, not real openings of the glyphs. Third, all ten retained components still admit closed outer contours. This does not mean that the thresholding defects leave the glyph shapes unchanged: the traced boundaries describe the selected binary mask, and the missing loop of `8` changes its topology. Hole counts therefore require inspection before they are used as recognition features.

The JSON file `task_5/outputs/task5_outlines.json` stores the complete ordered outer and hole traces, their starting pixels, and their full Freeman chain codes. All trace and start coordinates use $(\text{row}, \text{column})$ in the Task 3 sub-image. A non-degenerate chain includes the final step back to its start; a single-pixel hole is represented by its start and an empty chain. These data reproduce the boundary-pixel mask without re-running the follower. The earlier chain-code prefix and statistics remain available as compact summaries.

7 Task 6: Character Segmentation and Labelling

7.1 Objective

Task 6 requires segmenting the characters so that the different characters are separated as clearly as possible and each one is individually labelled. The input is the binary image selected in Task 4, in which the ten characters of `HD44780A00` already appear as ten separate bright regions on a dark background. The segmentation therefore has three steps: remove the residual noise, label every remaining connected component, and order the ten labelled characters from left to right.

7.2 Segmentation Method

7.2.1 Connected-component labelling

Foreground components were labelled with the classic two-pass union-find algorithm [1], implemented in `common.py`. In the first pass the image is scanned row by row; every foreground pixel takes the smallest label among its already-visited 8-neighbours, and the corresponding labels are merged in a union-find structure. In the second pass every label is replaced by the root of its equivalence class. The result is a label image in which each 8-connected foreground region has a unique label.

7.2.2 Noise removal by area

Task 4 reported 126 components, of which 10 are characters and 116 are texture residues. The largest noise component has 224 pixels while the smallest character has $2{,}719$ pixels, so an area criterion removes all noise without touching any character:

$$\text{keep component } C \iff |C| \geq 300. \tag{7.1}$$

The threshold is deliberately placed in the wide gap between the two populations, which makes the choice insensitive to small image changes.

7.2.3 Left-to-right ordering

The surviving components are sorted by their leftmost column so that label 1 is the leftmost character and label 10 the rightmost, matching the reading order of `HD44780A00`.

7.3 Results

figure 7.1 shows the ten labelled characters over the grayscale sub-image, and figure 7.2 adds the bounding box of every label. The boxes are 79–88 pixels wide and 114–131 pixels high, so the characters are remarkably uniform in size. figure 7.3 shows the ten extracted character crops that are saved for the classification stage.

7.4 Why Component Labelling Rather than Projection Splitting

The horizontal gaps between adjacent bounding boxes are only 9, 4, 10, 10, 2, 3, 3, 2, and 6 pixels, so several characters almost touch. A purely projection-based splitter would therefore be fragile. The effect is visible when the same labelling is applied to the global-Otsu binary from Task 4: there the `D` and first `4` merge, the `8` and `0` merge, and the two final `0`s merge, leaving only 7 groups instead of 10 (figure 7.4). The adaptive binary keeps the strokes separated, so the simple component-labelling segmentation already returns exactly one region per character, and no further de-merging or split heuristics are needed.

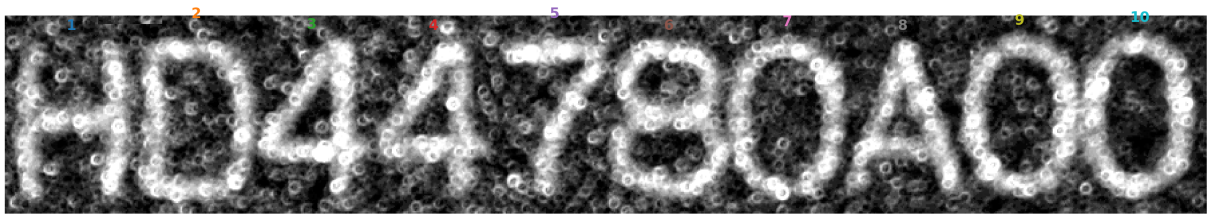


Figure 7.1: Segmented characters labelled 1–10 in reading order.

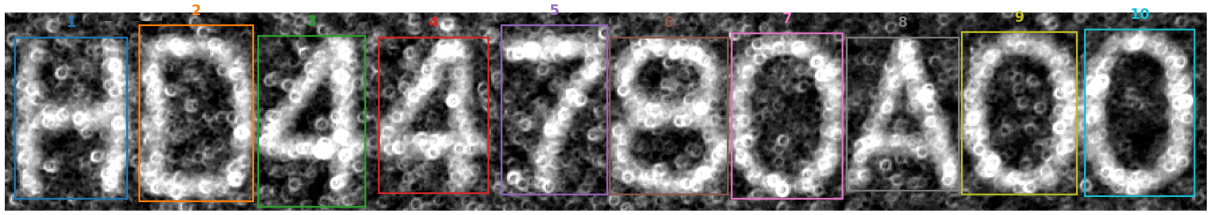


Figure 7.2: Bounding boxes of the ten segmented characters.

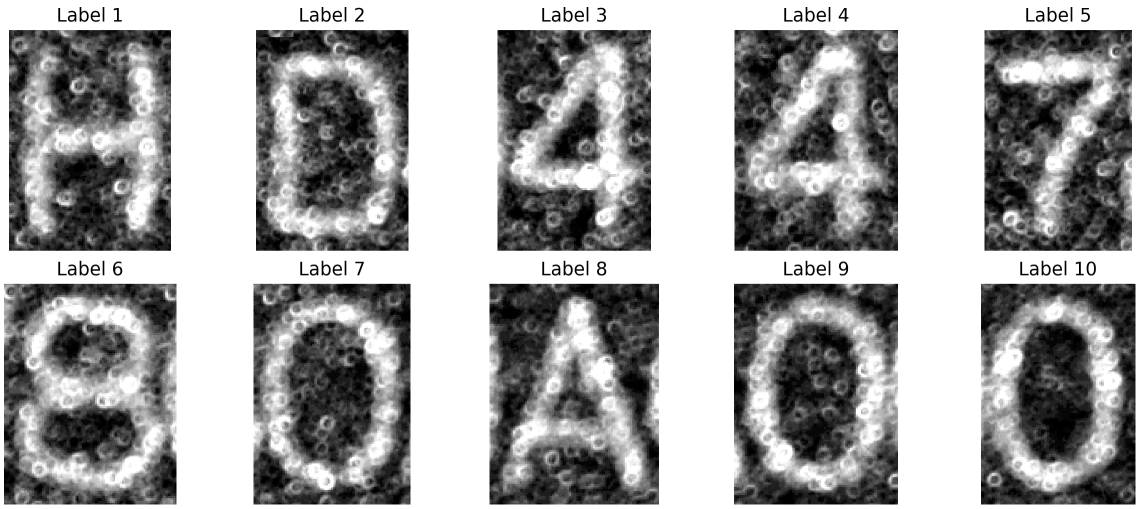


Figure 7.3: The ten extracted character crops (labels 1–10).



Figure 7.4: Character groups found on the global-Otsu binary (7 groups) and on the selected adaptive binary (10 groups). Merged neighbours are visible as boxes that span two characters.

Table 7.1: Segmentation result. Bounding boxes are given in sub-image pixel coordinates.

Label	Glyph	Bounding box (x_0, y_0)–(x_1, y_1)	Size ($w \times h$)	Area (px)
1	H	(8, 19)–(91, 138)	84×120	3836
2	D	(101, 10)–(185, 140)	85×131	4149
3	4	(190, 18)–(269, 144)	80×127	3485
4	4	(280, 19)–(361, 134)	82×116	3269
5	7	(372, 10)–(450, 135)	79×126	2719
6	8	(453, 19)–(540, 135)	88×117	4336
7	0	(544, 16)–(626, 138)	83×123	3859
8	A	(630, 19)–(713, 132)	84×114	3103
9	0	(716, 15)–(801, 135)	86×121	3980
10	0	(808, 13)–(889, 136)	82×124	3997

7.5 Analysis

The segmentation is exact for this row: after noise removal exactly ten components remain, and reading the components left to right reproduces `H D 4 4 7 8 0 A 0 0`. The result inherits two properties from the binarization stage. First, the `8` remains one component even though its bottom loop is partly open (Task 5), because the stroke itself is still connected; area filtering does not break it further. Second, the three `0`s have slightly different areas ($3{,}859$, $3{,}980$, $3{,}997$ pixels) because of local stroke-thickness variation, which shows why a fixed area interval rather than a single expected size is used to accept a component.

The cleaned binary is saved as `task_6/outputs/task6_cleaned_binary.png`, and one padded grayscale and binary crop per character is saved under `task_6/outputs/chars/` together with `task_6/outputs/task6_components.json`. These crops are the natural input of the CNN and non-CNN classifiers in Task 7, whose ground-truth label order is exactly the known string `HD44780A00`.

8 Task 7: Character Classification

8.1 Objective and experimental design

Task 7 compares a convolutional classifier (Subtask 7-1) with the requested non-convolutional supervised regression system (Subtask 7-2). The seven classes are `0`, `4`, `7`, `8`, `A`, `D`, and `H`. The implementation is in `task_7/task_7.py`; all numerical results, split manifests, checkpoints, confusion matrices, and tuning curves are saved in `task_7/outputs/`. The supplied Figure 1 image and the ten crops made in Task 6 are used only after training for inference, so no target-row pixels can leak into the learned parameters.

8.2 Data split and leakage control

There are 1,016 grayscale 128×128 PNG images in each class, giving 7,112 labelled samples. The base random seed is 20261013; deterministic per-candidate and per-member seeds are derived from it as specified below. The outer split is exactly 75% training and 25% test: 762 images per class are assigned to the outer training pool and 254 images per class to the untouched test set. Ten percent of the outer training pool (76 images per class) is held out as validation for model selection, leaving 686 images per class for the tuning fit. After selecting the input resolution and learning rate, the final models are refitted on all 5,334 images in the 75% outer training pool.

The file names contain repeated acquisition frames. A random file-level split would therefore

allow identical images to occur on both sides of the evaluation boundary. The script hashes decoded grayscale pixels with SHA-256, keeps every exact duplicate group in one partition, and solves a small subset-sum problem per class to retain the requested 762/254 counts. The resulting train/validation/test counts are 4,802/532/1,778; the three pairwise exact-pixel digest intersections are all zero. The complete audit is in `task_7/outputs/split_summary.json` and the reproducible assignment of every file is in `task_7/outputs/split_manifest.csv`.

8.3 Common preprocessing

Images are converted to one channel, resized with bilinear interpolation to a candidate square side (32 or 48 pixels), and divided by 255. The mean and standard deviation are fitted only on the tuning-train subset and the same statistics are applied to validation and test samples:

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}. \tag{8.1}$$

No augmentation is used because the seven glyphs are already centred and small geometric changes can alter the distinguishing strokes. The selected normalisation statistics are recorded in `task7_metrics.json`.

8.4 Subtask 7-1: CNN classifier

The CNN uses two 3×3 convolution–BatchNorm–ReLU layers with 32 channels, max pooling, two corresponding layers with 64 channels, and one 128-channel convolution followed by global average pooling. A linear layer maps the pooled feature to seven logits. Cross-entropy is used for the classification loss. Global average pooling makes the classifier insensitive to small translations while keeping the number of parameters at 140,071. The forward path is

$$1 \times s \times s \;\to\; 32 \;\to\; 64 \;\to\; 128 \to \text{GAP} \to 7,$$

where s is the selected input side. BatchNorm layers are included after each convolution [4], and AdamW with weight decay 10^{-4} is used for fitting [5].

8.5 Subtask 7-2: MLP one-hot regression

The non-CNN method is a fully connected multi-output regression model. For a class c, the target is the one-hot vector $y \in \{0, 1\}^7$ and the network outputs seven real-valued scores $f_\theta(x)$. It is trained with mean square error,

$$\mathcal{L}_{\text{reg}}(\theta) = \frac{1}{7} \left\| f_\theta(x) - y \right\|_2^2, \qquad \widehat{c} = \arg\max_j f_{\theta,j}(x). \tag{8.2}$$

This explicitly implements the requested supervised regression formulation; the MLP does not use convolution or a cross-entropy classification head. Its structure is

$$s^2 \;\to\; 512 \;\to\; 256 \;\to\; 7,$$

with GELU activations, BatchNorm1d and Dropout(0.25) after both hidden linear layers [4, 6]. The selected model has 659,463 parameters. Three independently seeded MLPs are fitted and their seven-dimensional regression outputs are averaged before the argmax decision. This ensemble reduces variance without changing the non-CNN nature of the method.

8.6 Hyperparameter and preprocessing investigation

The validation grid contains both 32×32 and 48×48 resizing and learning rates 10^{-3} and 3×10^{-4}. All eight runs are listed in table 8.1, and their validation curves are shown in figure 8.1.

Every tuning run uses a single network; the three-member MLP ensemble is fitted only after its configuration is selected. The complete machine-readable grid is in `task_7/outputs/tuning_results.csv`.

Table 8.1: Complete single-network validation grid. Bold configurations are selected for final fitting. CNN loss is cross-entropy; MLP loss is one-hot MSE, so the two loss scales should not be compared directly.

Method	Input	Initial LR	Best epoch	Val. accuracy	Val. loss
CNN	32×32	10^{-3}	18	0.9962	0.0157
CNN	32×32	3×10^{-4}	7	0.9887	0.1413
CNN	**48×48**	**10^{-3}**	**15**	**0.9981**	**0.0416**
CNN	48×48	3×10^{-4}	18	0.9868	0.0962
MLP regression	32×32	10^{-3}	18	1.0000	0.0055
MLP regression	**32×32**	**3×10^{-4}**	**17**	**1.0000**	**0.0079**
MLP regression	48×48	10^{-3}	17	1.0000	0.0046
MLP regression	48×48	3×10^{-4}	16	0.9981	0.0074

Configuration selection maximises validation accuracy. An exact tie is resolved first by the smaller input side and then by the smaller learning rate. Thus the MLP's three 100% validation runs are tied; selecting 32×32 and 3×10^{-4} follows this deterministic rule, rather than showing that this resolution is more accurate. Within a run, the best epoch maximises accuracy and uses the lower validation loss to break an accuracy tie.

For the CNN, increasing the side from 32 to 48 raises validation accuracy from 99.62% to 99.81% at initial learning rate 10^{-3}, but lowers it from 98.87% to 98.68% at 3×10^{-4}. The observed resolution effect therefore depends on the learning rate. Initial rate 10^{-3} outperforms 3×10^{-4} for both CNN resolutions within this budget; MLP differences are small and three settings reach the validation ceiling. These are exploratory grid results, not isolated causal estimates: each configuration is run once, and its seed also changes according to `base + 100*side + int(100000*lr) + model_offset`, with offsets 0 for CNN and 1 for MLP. Random initialisation and batch order can contribute to the small differences; repeated runs with shared seed sets would be needed to quantify sensitivity more reliably.

Table 8.2: Shared training settings and the final-fit schedule.

Setting	Value
Optimiser	AdamW; $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$; weight decay 10^{-4}
Batch sizes	128 for fitting, 256 for validation/test; no dropped final batch
Tuning budget	At most 18 epochs; early-stop patience 5 without a better accuracy/loss checkpoint
Tuning LR schedule	ReduceLROnPlateau on validation accuracy; factor 0.5, patience 2, minimum initial LR/20
Final fitting	All 5,334 outer-training samples; 15 CNN epochs or 17 epochs per MLP member; cosine LR decay with $T_{\max}$ equal to that epoch count
Final seeds	CNN 20268013; MLP members 20268014, 20268051, 20268088
Normalisation	At 32 px: $\mu = 0.8240095$, $\sigma = 0.3436231$; at 48 px: $\mu = 0.8239762$, $\sigma = 0.3558236$; fitted only on tuning-train
Loader/reproducibility	0 workers; deterministic cuDNN; CUDA selected if available

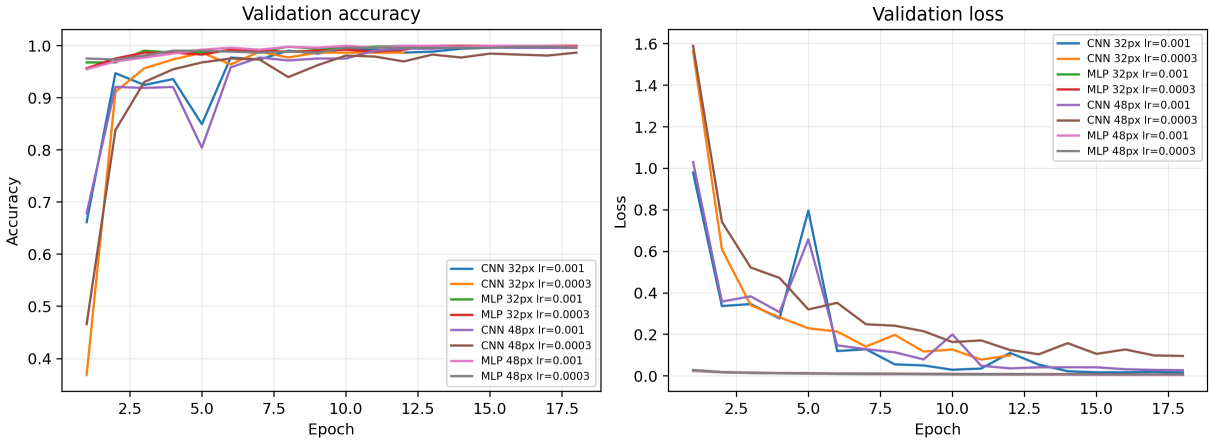


Figure 8.1: Validation accuracy and loss for the resolution and learning-rate investigation.

8.7 Held-out results and efficiency

The untouched 1,778-image test set gives the results in table 8.3. Both methods generalise well, but the CNN is 0.62 percentage points more accurate and has a slightly higher macro-F1. The MLP ensemble has a larger parameter count because every pixel connects to the first dense layer; nevertheless, its lower-resolution input makes batched inference faster in this small experiment. Training time includes the final fit (one CNN or three MLP members) on the complete outer training pool.

Table 8.3: Final held-out test performance and measured efficiency. Parameter counts are per member; training time covers the complete ensemble.

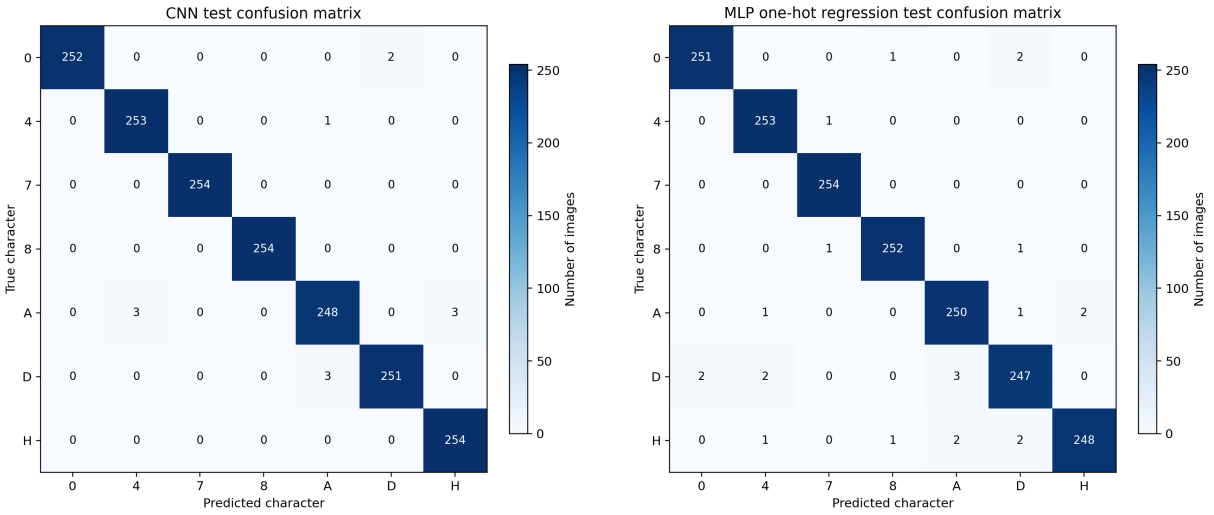
Method	Accuracy	Macro-F1	Params	Members	Train (s)	Infer. (ms/img)
CNN (48×48)	0.9933	0.9932	140,071	1	4.73	0.0375
MLP reg. ensemble (32×32)	0.9871	0.9871	659,463	3	7.19	0.0142

The MLP test regression MSE is 0.0084 before the argmax decision. Its confusion matrix, like the CNN matrix, is nearly diagonal; the largest residual confusions involve the visually similar `A`/`D`/`H` strokes. The matrices are provided as report-ready figures in figure 8.2.

8.8 Inference on the Figure-1 character row

After the held-out evaluation, the saved checkpoints were applied to the ten Task 6 character crops. None of these crops enters the split manifest, normalisation fitting, network optimisation, or hyperparameter selection. The original grayscale baseline inverts the bright-on-dark photograph, square-pads each crop, resizes it, and applies the training normalisation. It gives only 10% CNN character accuracy and 90% MLP character accuracy on the known ten-character reference row (table 8.4). This large CNN gap from the clean-dataset test score motivates a separate preprocessing diagnosis.

A single class-agnostic alignment rule was fixed before inspecting its new predictions: use the Task 6 binary crop, keep its largest 8-connected foreground component, crop the component's tight bounding box, invert to black strokes on white, and resize this box to the median relative foreground width and height measured on the 4,802 tuning-train images. Those medians are 0.609375 and 0.7578125; source foreground is defined by pixel value below 128. The resized glyph is centred on a white square canvas and then receives the original trained model's normalisation. This rule removes photograph texture and aligns glyph scale; it can also distort aspect ratio, since width and height are matched separately. No target labels, model scores,



(a) CNN **(b)** MLP regression ensemble

Figure 8.2: Confusion matrices on the duplicate-isolated held-out test set.

or held-out test measurements select this rule or its parameters. The grayscale baseline and fixed binary alignment are shown in figure 8.3.

Table 8.4: Figure 1 predictions, with ten ordered positions made explicit. Accuracy is measured on this single reference row after inference.

Position	1	2	3	4	5	6	7	8	9	10	Accuracy
Reference	H	D	4	4	7	8	0	A	0	0	—
CNN, gray	A	A	A	A	A	A	A	A	A	A	1/10 (10%)
MLP ensemble, gray	H	D	4	4	7	H	0	A	0	0	9/10 (90%)
CNN, aligned binary	A	D	4	4	7	8	0	A	8	0	8/10 (80%)
MLP ensemble, aligned binary	H	D	4	4	7	8	0	A	0	0	10/10 (100%)

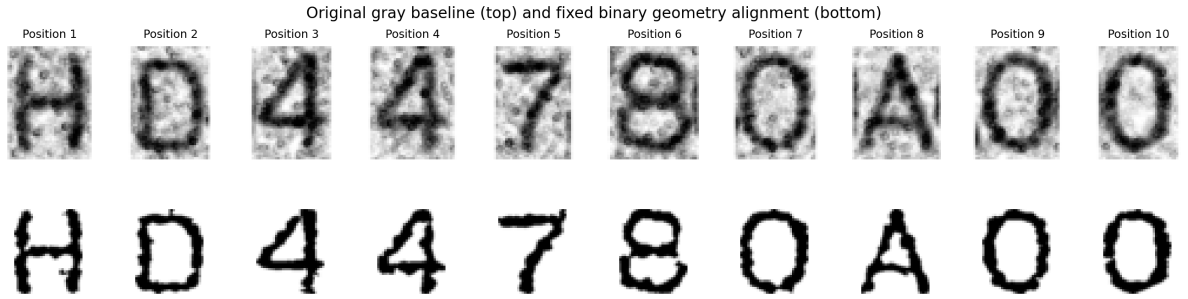


Figure 8.3: The original inverted grayscale inputs (top) retain background texture. Fixed binary foreground and training-median geometry alignment (bottom) produce cleaner, centred inputs without fitting on Figure 1.

The unchanged checkpoints predict `AD44780A80` with the aligned CNN and `HD44780A00` with the aligned MLP ensemble. The CNN still confuses the first `H` with `A` and the ninth `0` with `8`. These are exploratory results on one ten-character row, not a new held-out benchmark or evidence of general target-domain accuracy. The controlled clean-dataset test results and measured training times above are unchanged. The results support improving foreground preparation before adding ensemble members; additional evidence would require separately acquired target-domain samples. Any subsequent domain-aware retraining should use extra labelled training

images or augmentations of the allowed training partition, while keeping Figure 1 and the held-out test partition out of fitting and selection. The saved `target_predictions.json` records both preprocessing paths, per-character scores, geometry-fit provenance, and checkpoint files.

8.9 Reproduction and environment

The code uses Python and PyTorch; no scikit-learn or pretrained model is used. The verified runtime contains Python 3.8, PyTorch 2.0.0+cu118, torchvision 0.15.1+cu118, NumPy 1.24.2, Pillow 9.4.0, and Matplotlib 3.7.1, running on an NVIDIA GeForce RTX 4080 SUPER. The complete installation, Tasks 1–6 dependencies, full and smoke experiment commands, and saved-model `--inference-only` instructions are provided in section 9. Separate output directories preserve the reported experiment during reproduction. The supplied `task_7/environment.yml` pins the required Python dependencies, while `task_7/outputs/task7_metrics.json` records the selected settings, held-out results, and both target preprocessing paths.

The implementation uses PyTorch modules for tensor operations and automatic differentiation [7], Pillow for image decoding/resizing [8], NumPy for split auditing and metrics [9], and Matplotlib for figures [10]. These external libraries are acknowledged here as required by the project brief; the data split, model definitions, regression target, metric calculation, and experiment orchestration are implemented in the submitted script.

9 Implementation and Reproduction Guide

The Python implementation is reproduced from the repository root (the directory containing `main.tex` and `common.py`) as follows. Each task has an entry-point script, a report fragment, implementation notes, and an `outputs/` directory.

1. **Check the supplied data.** Place `Test_Image_2026.bmp` in `dataset_2026/`, alongside the seven class directories `0, 4, 7, 8, A, D, H`. Each class contains 1,016 PNG images (7,112 images in total), with source dimensions 128×128. Keep the BMP and its derived crops outside the labelled directories: they are reserved for Figure 1 inference.
2. **Create the software environment.** Install Conda, then execute:

   ```
   conda env create -f task_7/environment.yml
   source "$(conda info --base)/etc/profile.d/conda.sh"
   conda activate cvtask
   ```

   If `cvtask` already exists, replace the creation command with `conda env update -n cvtask -f task_7/environment.yml`. The environment file pins Python 3.8, PyTorch 2.0.0, torchvision 0.15.1 [11], CUDA runtime 11.8, NumPy 1.24.2, Pillow 9.4.0, and Matplotlib 3.7.1. The measured runtime was Ubuntu 20.04, Python 3.8.10, and PyTorch 2.0.0+cu118 on an NVIDIA GeForce RTX 4080 SUPER. Tasks 1–6 run on the CPU. Task 7 automatically selects CUDA if available, otherwise the CPU; prefixing its command with `CUDA_VISIBLE_DEVICES=""` forces CPU execution. Timings are device-dependent. Plots use the Agg backend and require no display server.
3. **Run every image-processing stage in order.**

   ```
   python task_1/task_1.py
   python task_2/task_2.py
   python task_3/task_3.py
   python task_4/task_4.py
   python task_5/task_5.py
   python task_6/task_6.py
   ```

   These scripts create or replace files in their fixed `task_N/outputs/` directories. Task 1 reads the BMP and saves `task1_percentile_stretch_p02_p98.png` for Tasks 2 and 3. Task 2 is

the filter-size comparison branch; Task 3 uses the Task 1 enhanced image and original BMP to save `task_3/outputs/task3_subimage.png`. Task 4 produces `task_4/outputs/task4_binary.png` and the Otsu comparison. Tasks 5 and 6 read the Task 3 grayscale image and Task 4 binary results. Task 5 writes outline figures and JSON contour records. Task 6 writes `task6_components.csv` and ten pairs of `task_6/outputs/chars/char_*_gray.png` and `char_*_bin.png` crops; these must exist before Task 7 to obtain the Figure 1 recognition results.

4. **Run the complete learning experiment.** Use a separate directory to preserve the report's supplied results:

```
python task_7/task_7.py --data dataset_2026 \
  --output task_7/outputs_reproduction
```

The default seed is 20261013, batch size 128, tuning limit 18 epochs, early-stopping patience 5, and data-loader workers 0. The full run performs the $32/48$-pixel and $10^{-3}/3\times10^{-4}$ learning-rate grid, chooses models on validation data, refits on the full 75% training pool using the selected validation epoch, then evaluates the 25% test partition and the target crops. It uses one CNN and three MLP regression members. For a short pipeline check, use:

```
python task_7/task_7.py --data dataset_2026 \
  --smoke --output task_7/outputs_smoke
```

Smoke mode uses one input size, one learning rate, at most two tuning epochs, and one MLP; it is not the reported experiment. Always specify the smoke directory because the default output is `task_7/outputs/`. The main artifacts are `split_summary.json` and `split_manifest.csv` (partition audit), `tuning_results.csv` and `tuning_curves.png` (model selection), `task7_metrics.json` and `method_comparison.csv` (results and timing), two confusion-matrix PNGs, `*.pt` checkpoints, and `target_predictions.json`. The report reads its formal figures from `task_7/outputs/`; independent reproduction and smoke runs leave these unchanged. Replacing the formal directory requires updating the report's numerical tables to match the new metrics. Exact numerical agreement and timing across different software or hardware are not guaranteed.
To repeat target inference from the supplied models without retraining, copy `task_7/outputs/` to a new `task_7/outputs_inference_check/` directory, then run:

```
python task_7/task_7.py --data dataset_2026 --inference-only \
  --output task_7/outputs_inference_check
```

This mode needs both crop variants, the dataset, split manifest, and copied checkpoints. It fits foreground geometry from training rows only and updates target inference while preserving test metrics and the grayscale baseline.

5. **Compile the report.** Install XeLaTeX, latexmk, and the recommended/extra LaTeX packages. On Ubuntu, run:

```
sudo apt-get install latexmk texlive-xetex texlive-latex-extra \
  texlive-latex-recommended texlive-fonts-recommended \
  curl cabextract fontconfig
```

Omit `sudo` when running as root. The provided font installer verifies and installs the actual Arial family (regular, bold, italic, and bold italic) under `~/.local/share/fonts/msttcorefonts/` by default, without root privileges; `REPORT_FONT_DIR` can override the directory. With all task figures present, execute:

```
bash scripts/install_report_fonts.sh
fc-match Arial
latexmk -xelatex -interaction=nonstopmode -halt-on-error \
  -file-line-error main.tex
```

The font check must resolve to Arial. The build also runs BibTeX for the references. The output is `main.pdf`. The project `.latexmkrc` removes auxiliary build files after a successful build while retaining the PDF.

`README.md` contains the full directory dependency table, artifact inventory, version-check command, and build troubleshooting command.

10 Discussion and Conclusion

Tasks 1–6 taught us how image-processing functions relate to measurable image properties. Grayscale conversion and contrast transforms change intensity distributions; averaging reduces local variation but also weakens edge gradients. Row and column projections describe text location, while thresholding determines foreground connectivity. Contour following exposes boundaries and holes, and connected-component labelling uses adjacency, area, and bounding boxes to separate characters. These relationships explain why each stage must preserve the information needed by the next stage, rather than merely produce a visually smoother or brighter image.

The comparisons support the selected pipeline. In Task 1, percentile stretching gave better character-to-background separation than histogram equalisation, which amplified texture; gamma correction produced a smaller improvement. Task 2 showed the noise–detail trade-off: larger averaging kernels suppressed more noise but blurred strokes, so 5×5 was a practical compromise. In Task 3, band-adaptive refinement reduced the effect of texture on coarse projection-based localisation, and a safety margin preserved the complete row. In Task 4, smoothing and opening reduced Otsu noise but left only seven merged character groups; local adaptive thresholding retained ten. Tasks 5 and 6 showed that correct component counts do not guarantee intact glyph topology: the open lower loop of `8` and tiny holes in `0` remained visible in the contours. Area filtering removed the much smaller noise regions, enabling ten separately labelled crops.

Our investigations also covered the mathematical definitions of the methods: grayscale weighting, percentile and gamma transforms, cumulative-histogram mapping, neighbourhood averaging, projection sums, threshold criteria, morphological operations, contour traversal, and component equivalence. The preceding task sections document these formulations and implementation rules. We examined how NumPy supports array operations and statistics, Pillow handles image decoding and resizing, Matplotlib presents results, and PyTorch supplies networks, losses, automatic differentiation, and optimisation [7–10]. The network investigation included standardisation, BatchNorm, Dropout, one-hot MSE regression, cross-entropy, and the actual hidden-layer activations:

$$\mathrm{ReLU}(z) = \max(0, z), \qquad \mathrm{GELU}(z) = z\Phi(z) = \frac{z}{2}\left[1 + \mathrm{erf}\left(\frac{z}{\sqrt{2}}\right)\right].$$

Here Φ is the standard normal cumulative distribution function. The CNN uses ReLU's hard zero cutoff; the MLP uses GELU's smooth weighting [12]. Both have linear output layers, with argmax used for the final class decision. These are investigated design choices; their individual benefits were not isolated through activation or regularisation ablation experiments.

Task 7's CNN achieved slightly higher held-out accuracy than the MLP ensemble (99.33% versus 98.71%). The ensemble took longer to train, while its lower-resolution input gave faster measured inference in this experiment. Resolution and learning-rate effects depended on the configuration, and single-run results limit the strength of sensitivity claims. On Figure 1, training-derived foreground alignment improved CNN recognition from one to eight correct characters and MLP recognition from nine to ten, using unchanged models. This illustrates the importance of matching foreground appearance and scale to the training data. The project therefore achieved a complete processing and recognition pipeline, while also showing that high accuracy on clean data does not ensure reliable recognition of textured photographs; the ten-character target check requires broader validation before general claims can be made.

References

[1] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Pearson, 4 edition, 2018.

[2] Nobuyuki Otsu. A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1):62–66, 1979. https://doi.org/10.1109/TSMC.1979.4310076.

[3] Abeer George Ghuneim. Contour tracing: Moore-neighbor tracing. ImageProcessingPlace tutorial. https://www.imageprocessingplace.com/downloads_V3/root_downloads/tutorials/contour_tracing_Abeer_George_Ghuneim/moore.html. Accessed 13 October 2026.

[4] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 448–456, 2015. https://arxiv.org/abs/1502.03167.

[5] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. https://arxiv.org/abs/1711.05101.

[6] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014. https://jmlr.org/papers/v15/srivastava14a.html.

[7] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019. https://pytorch.org/.

[8] Pillow contributors. Pillow documentation. Official documentation. https://pillow.readthedocs.io/en/stable/. Accessed 13 October 2026.

[9] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020. https://numpy.org/.

[10] John D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. https://matplotlib.org/.

[11] PyTorch contributors. Torchvision documentation. Official documentation. https://pytorch.org/vision/stable/. Included in the supplied environment; not required by the network implementation. Accessed 13 October 2026.

[12] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). arXiv:1606.08415, 2016. https://arxiv.org/abs/1606.08415.